

# Collaborative Filtering Assignment 2

Aditya Upadhyay (2022040)

February 18, 2026

## 1 Introduction

In this assignment, I am required to build a recommender system that predicts user ratings using only the user-item interaction matrix ( $I$ ) from the MovieLens 100K dataset. The problem is formulated as a multi-class classification task, where the goal is to predict a rating value from 1 to 5 for a given user-item pair. No external metadata or side information is used, so the model learns entirely from ( $I$ ). I constructed a bipartite graph based on the interaction matrix, implement a Graph Convolutional Matrix Completion (GCMC) model to learn latent representations of users and items, and evaluate the model using the provided test splits for each fold. Regularization techniques and model ensembling are applied to improve generalization while strictly relying only on ( $I$ ). I trained several ML models also to get a variable baselines that what goal should be LightGBM performed quite good.

## 2 Dataset and Problem Formulation

### 2.1 Data Characteristics

The MovieLens 100K dataset serves as the foundation for this analysis. It consists of 100,000 ratings applied to 1,682 movies by 943 users. The data sparsity is a defining characteristic of this domain. If every user had rated every movie, the interaction matrix would contain  $943 \times 1682 = 1,586,126$  entries. With only 100,000 recorded interactions, the density of the matrix is approximately 6.3%.

### 2.2 Evaluation Protocol

To ensure the statistical significance of our results and avoid overfitting to a specific subset of the data, we employed a strict **5-Fold Cross-Validation** strategy. The dataset was partitioned into five distinct training and testing sets ( $u1.base/u1.test$  through  $u5.base/u5.test$ ). All metrics reported in this document represent the average performance across these five folds.

## 2.3 Target Variable

The target variable  $y$  is the rating integer. Unlike regression approaches that might predict a rating of 3.7, our classification approach treats ratings as distinct classes. This is particularly useful for capturing non-linear user behaviors, such as a user who only gives 1s and 5s but never intermediate ratings.

## 3 Methodology

The methodology adopts a hybrid feature engineering approach followed by a comparative modeling phase. The process is strictly devoid of metadata, relying instead on mathematical transformations of the interaction graph.

### 3.1 Baseline Bias Model

Before deploying complex algorithms, it is essential to establish a baseline. We utilize a regularized bias model. The predicted rating  $\hat{r}_{ui}$  is modeled as the sum of the global average, a user-specific deviation, and an item-specific deviation:

$$\hat{r}_{ui} = \mu + b_u + b_i \quad (1)$$

Where:

- $\mu$  is the global mean rating across all users and items.
- $b_u$  is the bias for user  $u$  (e.g., a critical user might have a negative bias).
- $b_i$  is the bias for item  $i$  (e.g., a universally acclaimed movie has a positive bias).

These biases are estimated using an iterative Alternating Least Squares (ALS) procedure with regularization to prevent overfitting on users or items with very few ratings.

### 3.2 Feature Engineering Strategy

To enable standard supervised learning algorithms (like Random Forest or MLP) to function on this data, we must transform the sparse user ID and item ID into dense feature vectors. We employed the following feature sets:

#### 3.2.1 1. Statistical Moments

For every user and item, we computed aggregate statistics from the training set:

- **User Mean/Count:** The average rating given by a user and the total number of ratings they have provided.

- **Item Mean/Count:** The average rating received by an item and its total popularity.

These features help the model distinguish between niche movies and blockbusters, as well as between casual viewers and power users.

### 3.2.2 2. Latent Factorization via SVD

To capture the implicit "taste" and "style" clusters within the data, I applied Truncated Singular Value Decomposition (SVD). The sparse rating matrix  $R$  is approximated by the product of low-rank matrices:

$$R \approx U\Sigma V^T \quad (2)$$

I selected a latent dimension of  $k = 60$ . This results in a 60-dimensional vector representing every user and a 60-dimensional vector representing every item.

- A user vector might unknowingly encode preferences for "Action" or "Romance" (latent dimensions).
- An item vector encodes the presence of these attributes in the film.

The dot product of these vectors roughly approximates the rating, but by feeding the raw vectors as features into non-linear classifiers (like XGBoost), we allow the model to learn complex, non-linear interactions between user tastes and item attributes that a simple dot product cannot capture.

## 3.3 Machine Learning Models

I evaluated a spectrum of models using the 'scikit-learn' ecosystem and specialized gradient boosting libraries.

### 3.3.1 Linear and Distance-Based Models

- **Logistic Regression:** Serves as a linear baseline.
- **SGD Classifier:** Optimized using Stochastic Gradient Descent, suitable for large-scale learning.
- **K-Nearest Neighbors (KNN):** Assumes that users with similar feature vectors will rate items similarly.

### 3.3.2 Tree Ensemble Methods

Tree-based models are particularly adapt at handling tabular data with varying scales (though we did standardize features).

- **Random Forest & Extra Trees:** These bagging methods construct hundreds of decor-related decision trees, reducing variance and resist overfitting.
- **Gradient Boosting (XGBoost, LightGBM, CatBoost):** These boosting methods build trees sequentially, where each new tree attempts to correct the errors of the previous ones. They optimize a differentiable loss function (Log Loss) and are known for state-of-the-art performance on structured data.

### 3.4 Deep Learning Architectures

I implemented two specific neural network architectures using TensorFlow/Keras to compare against the tree-based methods.

#### 3.4.1 Neural Collaborative Filtering (NeuralCF)

Unlike traditional Matrix Factorization which relies on a dot product, NeuralCF uses a Multi-Layer Perceptron (MLP) to learn the interaction function.

- **Inputs:** User ID and Item ID.
- **Embeddings:** These IDs are mapped to dense vectors via an Embedding layer (dimension 64).
- **Interaction:** The user and item vectors are concatenated (rather than multiplied).
- **Deep Layers:** The concatenated vector passes through dense layers (256  $\rightarrow$  128 neurons) with ReLU activation and Dropout (0.3) for regularization.
- **Output:** A softmax layer predicting the probability distribution over the 5 rating classes.

#### 3.4.2 Siamese Network Architecture (Sir told in class as a good model to work on this type of dataset)

Inspired by metric learning, this architecture attempts to minimize the "distance" between compatible user-item pairs.

- It processes User and Item embeddings through identical sub-networks (Siamese twin towers) to project them into a shared latent space.
- A specialized 'Lambda' layer computes the absolute difference  $|u - i|$  between the projected vectors.
- This difference vector, combined with the original projections, is fed into a classification head.

## 4 GNN Council

### 4.1 Graph Construction

The recommendation problem is modeled as a bipartite graph:

$$\mathcal{G} = (\mathcal{U} \cup \mathcal{V}, \mathcal{E})$$

where:

- $\mathcal{U}$  represents users,
- $\mathcal{V}$  represents items,
- $\mathcal{E}$  represents rating interactions.

Each rating value is treated as a distinct relation type. Since graph neural networks operate on directed graphs, we include both directions:

$$\text{User} \rightarrow \text{Item}, \quad \text{Item} \rightarrow \text{User}$$

Thus, the total number of relation types is:

$$5 \text{ ratings} \times 2 \text{ directions} = 10.$$

### 4.2 Model Architecture

The model follows the Graph Convolutional Matrix Completion (GCMC) framework, consisting of:

1. Relation-aware graph encoder (RGCN)
2. Bilinear decoder

#### 4.2.1 Embedding Layer

Each user and item is associated with a learnable embedding:

$$\mathbf{u}_i, \mathbf{v}_j \in \mathbb{R}^d$$

where  $d = 64$  in our implementation.

#### 4.2.2 Relational Graph Convolution

We employ two RGCN layers to propagate information across the bipartite graph.

For layer  $l$ , the update rule is:

$$\mathbf{h}_i^{(l+1)} = \sigma \left( \sum_{r \in \mathcal{R}} \sum_{j \in \mathcal{N}_r(i)} \frac{1}{c_{i,r}} \mathbf{w}_r^{(l)} \mathbf{h}_j^{(l)} \right)$$

where:

- $\mathcal{R}$  is the set of relations,
- $\mathcal{N}_r(i)$  denotes neighbors of node  $i$  under relation  $r$ ,
- $\mathbf{w}_r^{(l)}$  are relation-specific weight matrices,
- $\sigma$  is LeakyReLU activation.

Layer normalization and dropout are applied after each layer. A residual connection is added:

$$\mathbf{z} = \mathbf{h}^{(2)} + \mathbf{x}$$

This preserves original embedding information while incorporating neighborhood aggregation.

#### 4.2.3 Bilinear Decoder

Instead of a simple dot product, a relation-specific bilinear decoder is used:

$$s_r(u_i, v_j) = \mathbf{u}_i^\top \mathbf{Q}_r \mathbf{v}_j$$

To reduce parameter complexity, basis decomposition is applied:

$$\mathbf{Q}_r = \sum_{k=1}^B a_{rk} \mathbf{B}_k$$

where:

- $\mathbf{B}_k$  are shared basis matrices,
- $a_{rk}$  are rating-specific coefficients.

The final logits for classification are:

$$\text{logits} = [s_1(u_i, v_j), s_2(u_i, v_j), \dots, s_5(u_i, v_j)]$$

### 4.3 Loss Function

The model is trained using cross-entropy loss:

$$\mathcal{L} = - \sum_{c=1}^5 y_c \log(\hat{y}_c)$$

where  $y_c$  is the true one-hot label and  $\hat{y}_c$  is the predicted probability.

### 4.4 Regularization

To prevent overfitting:

- Dropout rate = 0.7
- Edge dropout rate = 0.4
- Weight decay =  $5 \times 10^{-4}$
- Gradient clipping at norm 1.0

Edge dropout randomly removes graph edges during training, improving robustness.

### 4.5 Optimization

We use the AdamW optimizer with learning rate 0.015. Learning rate scheduling is applied using ReduceLROnPlateau.

### 4.6 Ensemble Strategy

To reduce variance and improve generalization, an ensemble of three independently initialized models is trained per fold.

Final prediction is obtained by averaging logits:

$$\hat{y}_{\text{ensemble}} = \frac{1}{M} \sum_{m=1}^M \hat{y}^{(m)}$$

where  $M = 3$ .

### 4.7 Summary

The proposed approach integrates:

- Relation-aware graph convolution,

- Bilinear interaction modeling,
- Strong regularization,
- Model ensembling.

The method operates purely on interaction data and demonstrates the effectiveness of graph-based collaborative filtering without relying on external metadata.

## 5 Experimental Framework

### 5.1 Hyperparameter Optimization

To ensure a fair comparison, we employed **Optuna**, a state-of-the-art hyperparameter optimization framework. For key models (RandomForest, LightGBM, XGBoost), we defined search spaces covering:

- Number of estimators (trees): [100, 150, 200]
- Learning rates: [0.05, 0.1]
- Tree depth: Varies by model

Optuna uses a Tree-structured Parzen Estimator (TPE) to efficiently sample the hyperparameter space, focusing on promising regions rather than a brute-force grid search.

### 5.2 Metrics

We focus on two primary metrics:

1. **Accuracy:** The percentage of exactly correct rating predictions.
2. **Macro F1-Score:** The harmonic mean of precision and recall, calculated for each class (1-5) and then averaged. This is crucial because ratings are often imbalanced (users tend to give more 3s and 4s than 1s). Macro-F1 penalizes models that simply predict the majority class.

## 6 Results

The following table summarizes the aggregated performance across all 5 folds. The results are sorted by Model Type for clarity.



Table 1: Aggregated Performance Metrics (5-Fold CV)

| Category                   | Model Architecture             | Mean Accuracy | Mean Macro-F1 |
|----------------------------|--------------------------------|---------------|---------------|
| <i>Baselines</i>           | Bias Baseline                  | 0.4075        | 0.2668        |
|                            | Logistic Regression            | 0.4312        | 0.3616        |
|                            | SGD Classifier                 | 0.4061        | 0.3370        |
|                            | Complement Naive Bayes         | 0.4012        | 0.3120        |
| <i>Ensembles</i>           | Random Forest                  | 0.4429        | 0.4014        |
|                            | Extra Trees                    | 0.4411        | 0.3900        |
|                            | <b>LightGBM</b>                | <b>0.4479</b> | 0.3926        |
|                            | <b>XGBoost</b>                 | 0.4400        | <b>0.4034</b> |
|                            | CatBoost                       | 0.4350        | 0.3951        |
| <i>Neural Nets</i>         | Neural Collaborative Filtering | 0.4211        | 0.3014        |
|                            | Siamese Network                | 0.4192        | 0.3328        |
|                            | MLP (Scikit-learn)             | 0.4285        | 0.3550        |
| <i>GNN Council (Novel)</i> | <b>GNN Council</b>             | <b>0.4519</b> | <b>0.4050</b> |

## 6.1 Detailed Analysis of Results

### 6.1.1 Superiority of Gradient Boosting

The empirical data clearly indicates that Gradient Boosting Decision Trees (GBDTs), specifically LightGBM and XGBoost, provide the state-of-the-art performance for this dataset.

- **LightGBM** achieved the highest overall accuracy (**44.79%**). Its leaf-wise tree growth strategy allows it to model complex, non-linear decision boundaries within the latent feature space derived from SVD.
- **XGBoost** achieved the highest Macro-F1 score (**0.4034**). This indicates that XGBoost was better at balancing performance across all rating classes, particularly the minority classes (ratings of 1 or 2), rather than just optimizing for the most common ratings.
- **GNN Council** achieved overall highest accuracy among all the models, this is novel which i though getting idea from LLM Council to use this. It also beat all models in F1 making it robust. I added ML based in bold signifying max what i can get using ML techniques.

### 6.1.2 The "Tabular" Gap in Deep Learning

An interesting observation is that the specialized Neural Network architectures (NeuralCF and Siamese) did not outperform the tree-based ensembles. NeuralCF achieved an accuracy of 42.1%, lagging behind Random Forest (44.3%) by over 2 percentage points. This aligns with current literature suggesting that on structured, tabular datasets with limited samples (100k is considered small-to-medium in Deep Learning terms), ensemble trees

often outperform neural networks. Neural networks typically require vastly more data to learn the optimal feature interactions that trees can find greedily. To capture complex Relations between users and items , i then used GCN council.

### 6.1.3 Effectiveness of Feature Engineering

The baseline bias model achieved 40.7% accuracy. The jump to 44.8% with LightGBM represents a significant relative improvement. This gain is almost entirely attributable to the SVD latent features. The ability of the SVD to compress the user-item interaction graph into a dense 60-dimensional vector provided the classifiers with the context needed to understand user "taste" beyond simple average ratings.

## 7 Conclusion

This report presented a comprehensive evaluation of multi-class rating classification on the MovieLens 100K dataset. By systematically engineering latent features via Truncated SVD and applying a rigorous cross-validation protocol, we demonstrated that metadata is not strictly necessary to achieve robust predictive performance.

The key findings are:

1. **Gradient Boosting dominates:** LightGBM and XGBoost are the superior architectures for this tabular, interaction-based task.
2. **Neural Networks are not a silver bullet:** Without massive scale, specialized architectures like NeuralCF struggle to beat well-tuned ensemble trees on dense feature sets.
3. **Feature Engineering is King:** The transformation of sparse IDs into dense latent vectors (SVD) and statistical moments was the primary driver of model performance, contributing more to the final accuracy than the choice of classifier itself.
4. **GNN Council** If we utilizing a Council ensembling and modeling it as a graph problem it would tend to perform better which we can see and infer directly .

## 8 Refereces

1. M. Schlichtkrull, T. N. Kipf, P. Bloem, R. van den Berg, I. Titov, and M. Welling, "Modeling Relational Data with Graph Convolutional Networks," *European Semantic Web Conference (ESWC)*, 2018.
2. **NumPy** – Numerical computing library for array operations.  
<https://numpy.org/>
3. **Pandas** – Data manipulation and preprocessing.  
<https://pandas.pydata.org/>

4. **SciPy** – Scientific computing utilities and sparse matrix support.  
<https://scipy.org/>
5. **Scikit-Learn** – Machine learning framework used for classical models and evaluation metrics.  
<https://scikit-learn.org/stable/>
6. **Optuna** – Hyperparameter optimization framework.  
<https://optuna.org/>